KHUDA

Computer Vision

7주차

Attention & Transformer

트랙장 : 이승준

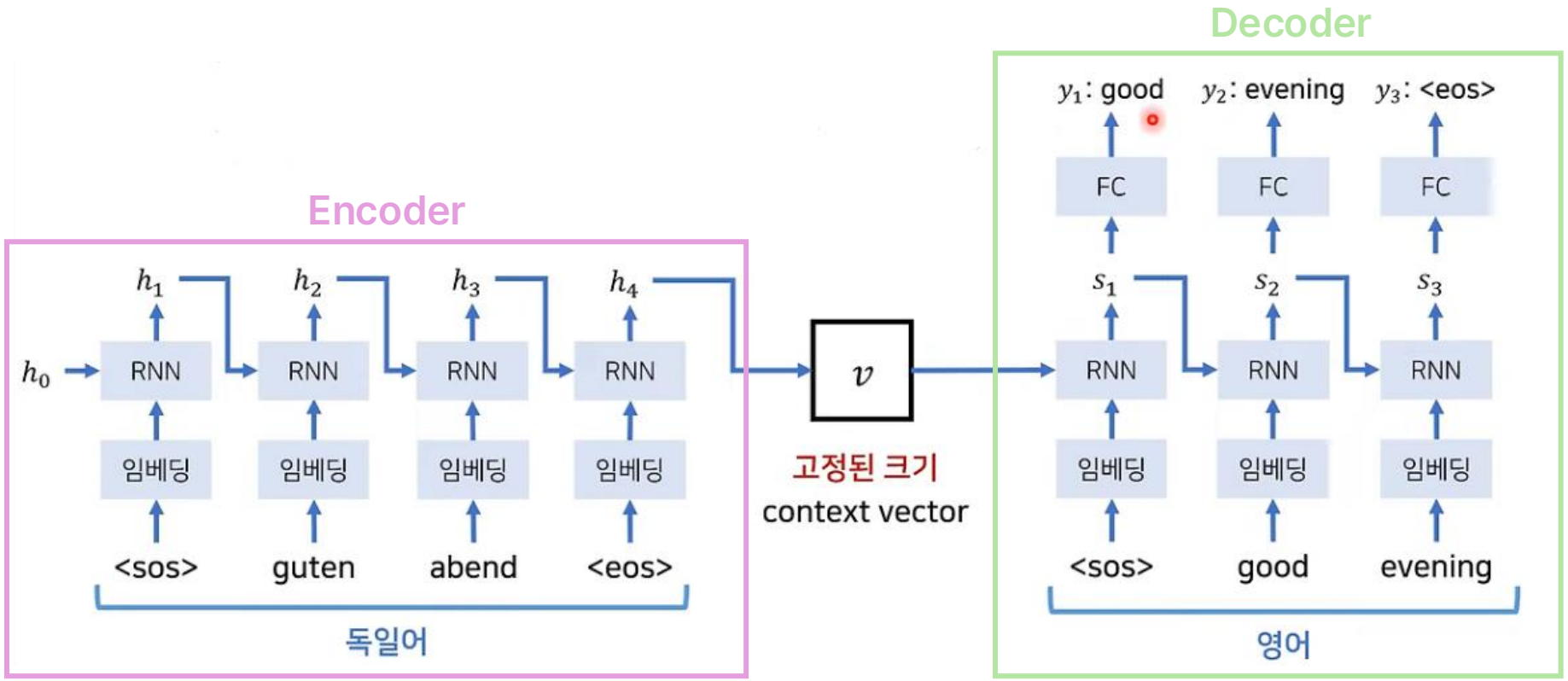
Attention

RNN 구조

Encoder : 입력 데이터의 정보를 잘 정리해서 Vector 형태로 만들어주는 녀석 (정보 저장 역할)

Decoder : 인코더에서 나온 Vector를 다른 형태의 결과로 해석하는 녀석 (정보 해석 역할)

즉, NLP에서 긴 문장을 입력으로 받아, 하나의 vector 형태로 변환하고 이를 통해 결과를 해석하는 구조임



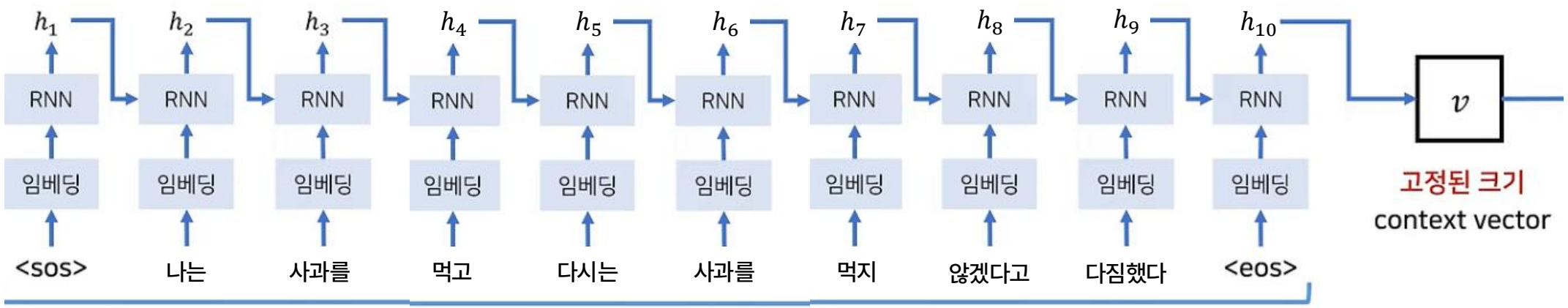
RNN의 단점

Encoder에 긴 문장이 입력으로 들어오면, 아무리 길더라도 하나의 vector로 정보를 함축해야 함.

따라서, 정보 손실이 생기는 문제가 발생함. 너무 많이 압축하다 보니 Bottleneck이 발생하는 것

이를 해결하기 위해선, 매 Hidden Layer를 모두 직접적으로 Decoder에 전달해서 예측에 반영할 수 있는 메커니즘이 필요함

-> 이걸 해주는게 Attention 메커니즘



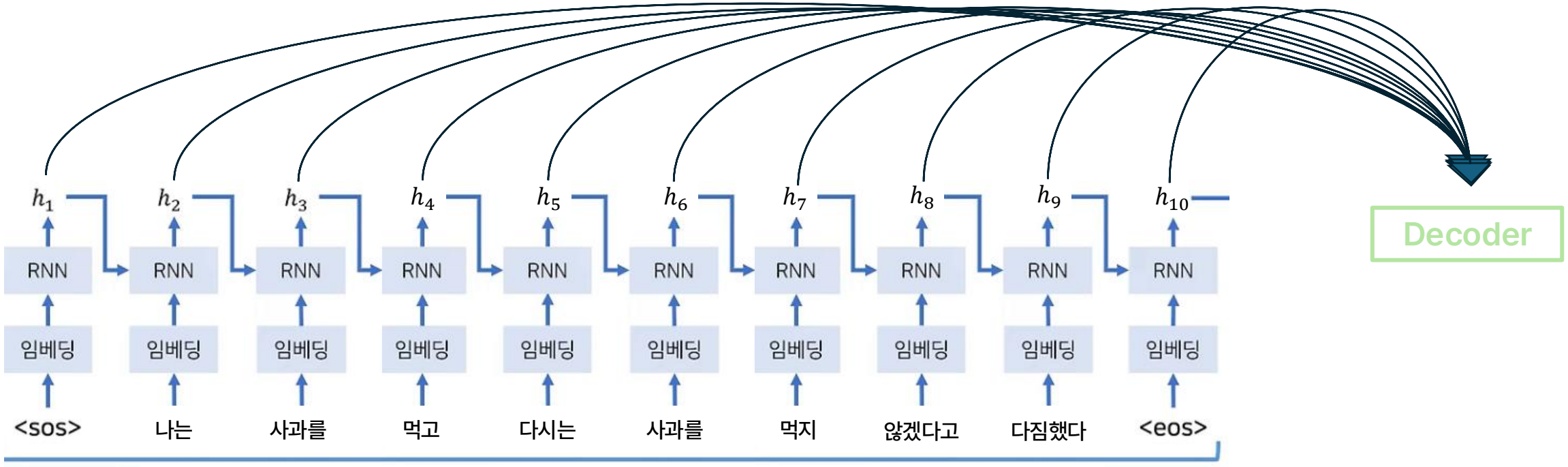
Attention 메커니즘

아래와 같이, 매 Hidden state Vector의 정보를 모두 디코더로 전달하되 어떤 단계의 정보를 더욱 반영할 것인지를 결정해야 함

단순히 모두 똑같은 비율로 전달하는게 아니라, Sequence 데이터니까 적절히 중요한 부분을 찾아서 반영해주자는 것임

근데 그 중요한 정도를 Input이 결정하는게 아니라, 지금 결과로 나가려는 녀석에게 직접 정할 수 있도록 기회를 주는 것임.

“Decoder에서 다음 단어를 예측으로 뽑으려고 하는데, 입력 중에 어디를 더 집중해서 봐야할까?” 라는 질문을 역으로 Input에 던지는 것



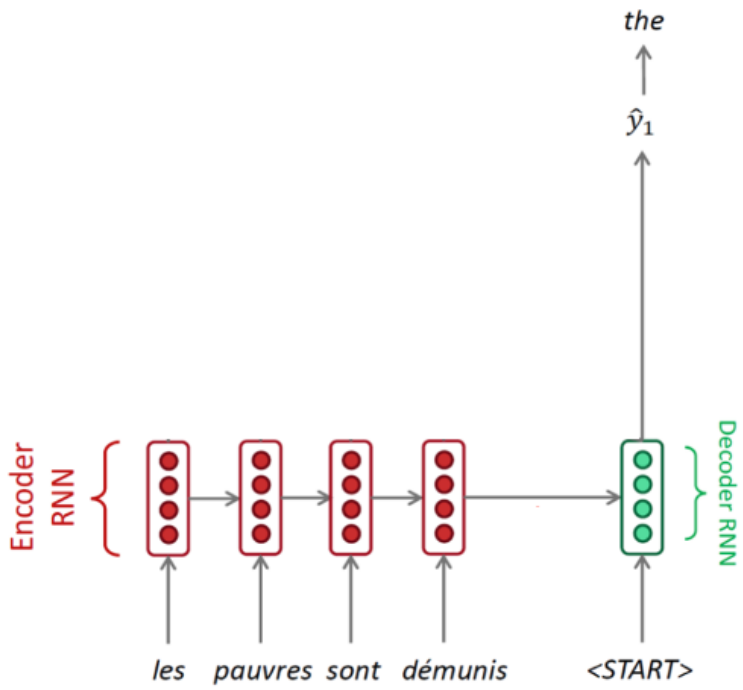
Attention

기본 RNN은 입력을 순차적으로 Encoding하고 다음 단계로 전달하여 최종 Hidden Vector를 Decoder에 넘기고, 이를 받아서 예측 진행

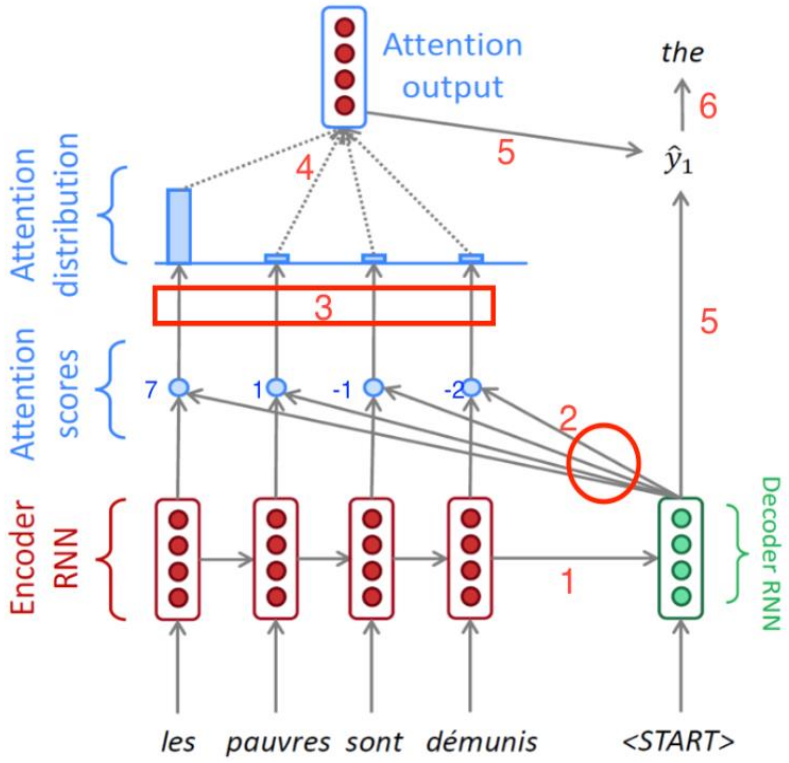
그러나 Attention은 Decoder에서 지금 입력받은 Hidden Vector가 어떤 Input과 가장 연관이 있는지, 어디를 한번 더 확인할지를 결정해서 중요한 Input의 정보를 한번 더 강조해서 예측을 수행한다고 생각하면 됨.

순서대로 Attention 메커니즘을 익혀봅시다

RNN



RNN + Attention

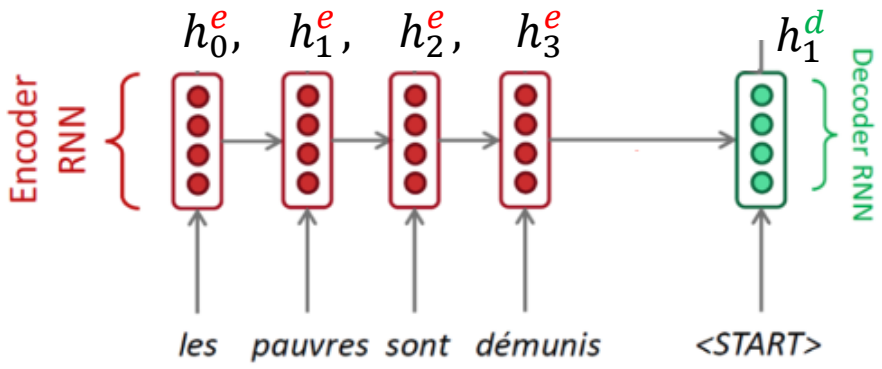


Attention

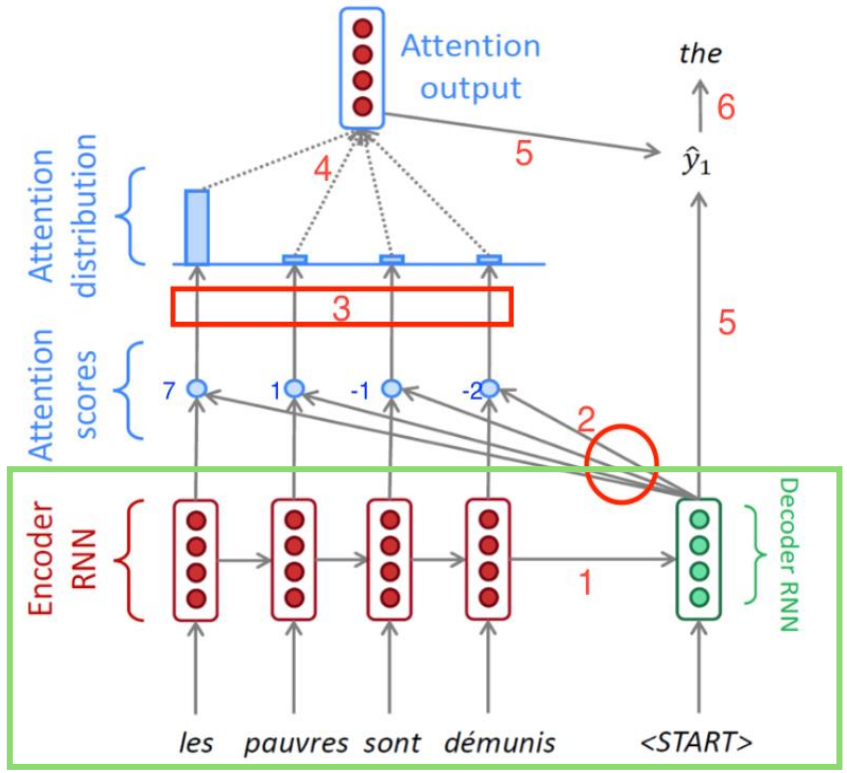
1. Input Encoding 및 Decoder에 전달

이 부분은 기존 RNN과 달라진게 없음. Sequence 데이터를 입력으로 받아서, $h_0^e, h_1^e, h_2^e, h_3^e$ Hidden Vector를 만들고, 이를 가공해서 Decoder에 전달 h_1^d

기본 RNN이라면 h_1^d를 통해 예측을 진행해야 함. 즉, 지금 단계는 "les"를 번역해야하니까, "the"라고 예측을 했을 것임



RNN + Attention



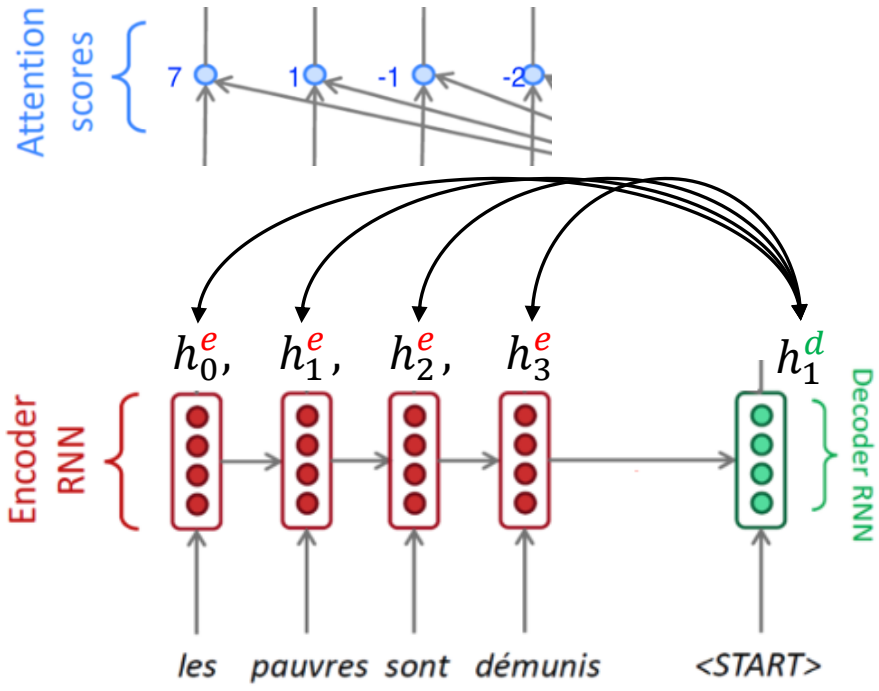
Attention

2. h_1^d를 통해 예측을 진행하는 것이 아니라, 지금 출력에 사용될 h_1^d이 Vector가 Input들 중에 무엇과 상관이 컸는지를 체크함.

명시적으로 지금 출력에 모든 Input들의 정보를 적당히 분배해서 추가해주는 과정이 추가되는 것임.

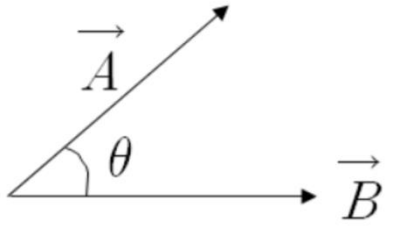
따라서, 과연 Input들(Hidden vector)을 어떤 비율로 정보를 추가해줘야 하는지를 판단하는 방법이 필요함. 바로 벡터의 내적을 하는 것임.

만약 벡터가 똑같은 방향을 바라본다면 = 벡터가 비슷하다면 내적 결과가 커질 것임. 이를 통해 $h_0^e, h_1^e, h_2^e, h_3^e$ 들 중에 어떤 녀석을 더 중요하게 봐야하는지를 찾음



지금은 "les"를 번역하는 단계니까, 당연히 h_1^d 는 "les"에서 나온 h_0^e 랑 가장 비슷할 것임

그게 Attention Score를 통해 확인되는 것임. 7 / 1 / -1 / -2



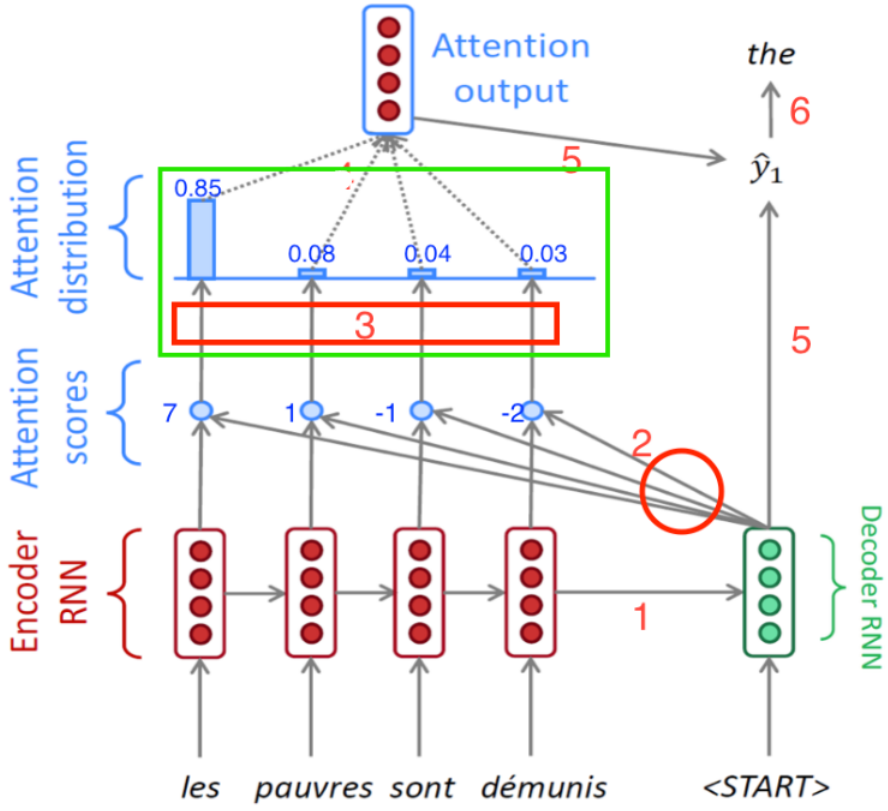
$$\vec{A} \cdot \vec{B} = |\vec{A}||\vec{B}|\cos\theta$$

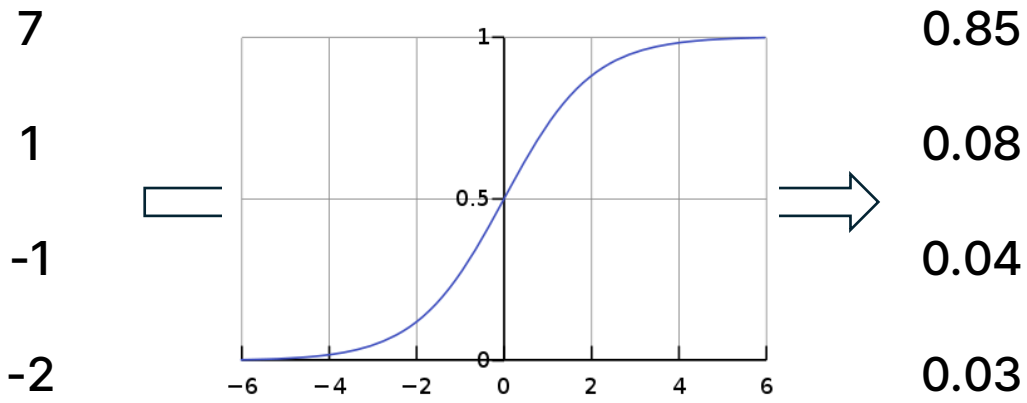
Attention

3. 계산된 Attention Score를 Softmax 함수를 통해 확률 값으로 변경함

벡터 방향이다 보니까, 내적을 하면 음수도 나오기 때문에 이를 확률값으로 변경함으로써 비율의 의미를 갖도록 변경해줌

이렇게 계산된 0.85 = h_0^e가 얼마나 중요한가 / 0.08 = h_1^e가 얼마나 중요한가 / 0.04 = h_2^e가 얼마나 중요한가 / 0.03 = h_3^e가 얼마나 중요한가





Attention

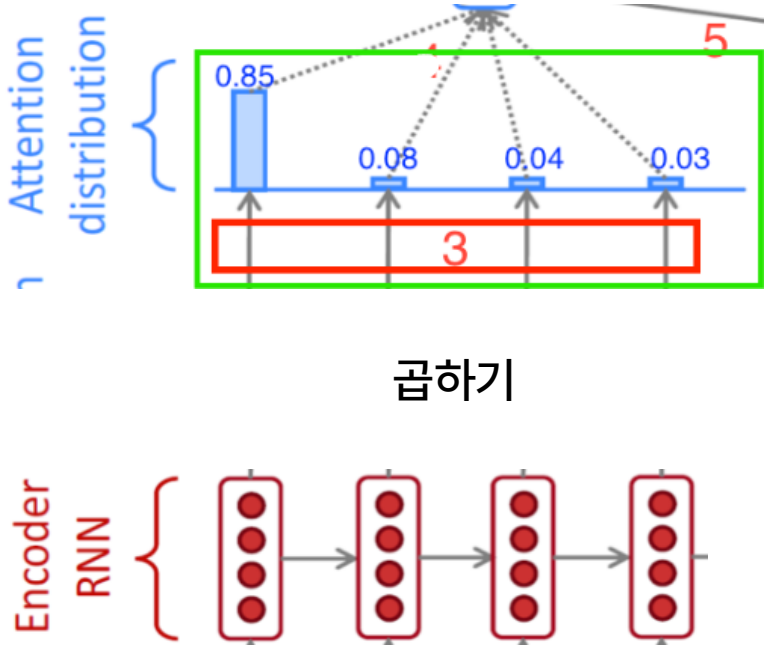
4. 이제 이러한 가중치를 반영한 Attention Output을 생성함.

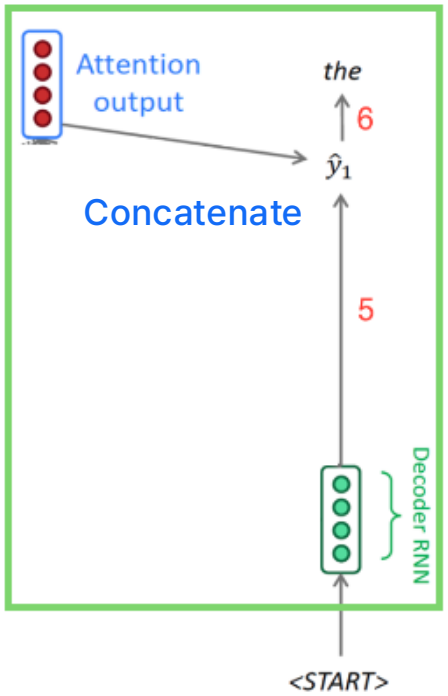
$$Attention\ Output\ Vector = 0.85 * h_0^e + 0.08 * h_1^e + 0.04 * h_2^e + 0.03 * h_3^e$$

이렇게 하면, 각 hidden vector의 영향이 훨씬 더 명시적으로 반영된 예측을 위한 Hidden Vector가 만들어짐.

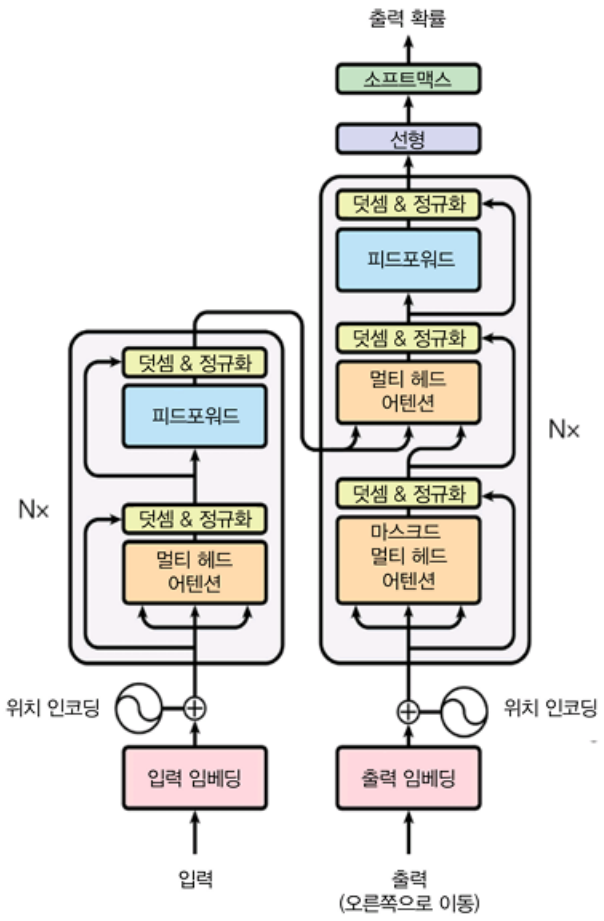
이걸 바로 쓰는게 아니라, 얘를 아까 봤던 h_1^d 에다가 Concatenate 해서 최종 예측을 진행하는 것임

즉 원래 예측에 쓰려고 했던 녀석(h_1^d) + 명시적으로 Input들과의 관계를 포함시켜준 녀석($Attention\ Output\ Vector$) -> 예측





Attention

RNN + Attention 메커니즘은 여전히 RNN을 통해 feature를 만들기 때문에 Long-term memory를 완벽하게 뽑아내지는 못함.

따라서 Transformer가 등장하게 됨